

Problem Set 3
COMP301 Spring 2025
Week 3: 03.03.2025 - 09.03.2025

Instructions:

- Submit your answers to the Blackboard PS3 assignment until **March 8th** Saturday, at 23.59.
- Please submit only **one single PDF file**, where all of your codes for each of the parts are included.
- Name your submission file as *id_username_ps3.pdf*
(Example: *00000_akutuk21_ps3.pdf*).

Problem 1: Given a list and a number n , implement a procedure named "repeatN" that repeats each element of the list n times and returns the new list. (Hint: you can use append instead of cons to achieve the results below)

```
(repeatN '(1 5 7) 2) ; returns (1 1 5 5 7 7)
(repeatN '(1 2) 5) ; returns (1 1 1 1 1 2 2 2 2 2)
```

Problem 2: Given a list, write a procedure called (sum-even list) by just using the higher order procedures map-filter-reduce such that the procedure returns the sum of all even numbers in a list.

```
(sum-even '(1 2 3)) ; returns 2
(sum-even '(1 2 3 4 5)) ; returns 6
```

Problem 3: (Challenging). In this part, you will implement your own higher-order procedure that will combine filter and reduce. First implement a procedure named "isIn?" that takes a list and a number n which will return true if the number is in the list, false otherwise. (If you want a challenge you can try to take the nested list into account as well. (ex: (isIn?-nested '(1 2 (3 4) 5) 3) $\rightarrow$ #t)). Secondly, implement the higher order procedure named "myProc", (myProc pred lst1 lst2 op init) such that the predicate (pred) needs to accept a list and an element and return a boolean. The procedure "myProc" returns the computed single result that is reduced while operation on the lst1 with op. (**Hint:** while using the pred give lst2 as it is and iterate over lst1)

```
(myProc isIn? '(1 5 8 13) '(1 2 3 4 5) + 0) ;returns 6 (1,5 isin the lst2 1+5=6)
(myProc isIn? '(1 5 8 13) '(1 5 10 12) * 1) ; returns 5
(myProc (lambda (ls2 n) (not (isIn? ls2 n))) '(1 5 8) '(1 2 5) + 0) ; returns 8
(myProc isIn?-nested '(1 5 8 13) '(1 (2 (3 4) 8)) / 1) ; returns 1/8
```

1. We decided to break the problem into smaller parts by defining a helper function. The helper function returns a small list with repeated elements, and the main function generates and concatenates these small lists.

```
(define (repeatN lst n) (if (null? lst) lst (append (appendN (car lst) n) (repeatN (cdr lst) n))))  
(define (appendN a n) (if (= n 0) '() (cons a (appendN a (- n 1)))))
```

2. We implemented reduce and filter as they were covered during the lectures. Then we created sum-even which uses these 2 functions.

```
(define (reduce base f lst) (if (null? lst) base (f (car lst) (reduce base f (cdr lst)))))  
(define (filter f lst) (cond  
  ((null? lst) lst)  
  ((even? (car lst)) (cons (car lst) (filter f (cdr lst))))  
  (else (filter f (cdr lst)))))
```

```
(define (sum-even lst) (reduce 0 + (filter (lambda (x) (= (remainder x 2))) lst)))
```

3. This is the isIn function

```
(define (isIn? lst n) (cond  
  ((null? lst) #f)  
  ((= (car lst) n) #t) (else (isIn? (cdr lst) n))))
```

4. This is isIn-nested, it is similar to isIn, but also uses "list?" function to execute correctly.

```
(define (isIn?-nested lst n) (cond  
  ((null? lst) #f)  
  ((list? (car lst)) (or  
    (isIn?-nested (car lst) n)  
    (isIn?-nested (cdr lst) n)))  
  ((= (car lst) n) #t)  
  (else (isIn?-nested (cdr lst) n))))
```

5. This is myProc. We also broke it into 2 parts for readability. The helper merges the lists, and the main function reduces the merged list.

```
(define (myProc pred lst1 lst2 op init)
  (reduce init op (myProcHelper pred lst1 lst2)))
(define (myProcHelper pred lst1 lst2) (cond
  ((null? lst1) lst2)
  ((pred lst2 (car lst1)) (cons
    (car lst1)
    (myProcHelper pred (cdr lst1) lst2)))
  (else (myProcHelper pred (cdr lst1) lst2))))
```